

M6 · SOLIDITY 심화 · S35

상속·인터페이스·라이브러리

inheritance · interface · library · abstract · receive / fallback

강의 50분 + 실습 10분

상속

인터페이스

라이브러리

COINCRAFT
ACADEMY

01 상속

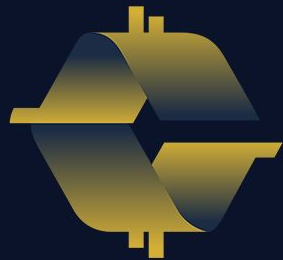
is · virtual · override · super

부모 컨트랙트의 모든 것이 자식에 포함된다

is 키워드

virtual / override

super



COINCRAFT
ACADEMY

상속 (1/3) — is 키워드

INHERITANCE — BASE CONTRACT & CHILD CONTRACT

Base Contract

```
contract Base {  
    address public owner;  
  
    constructor() {  
        owner = msg.sender;  
    }  
  
    function greet() public virtual  
        returns (string memory) {  
        return "Hello from Base";  
    }  
  
    modifier onlyOwner() {  
        require(msg.sender == owner);  
        _;  
    }  
}
```

Child Contract

```
contract Child is Base {  
    // owner, onlyOwner 자동 포함  
    // constructor 없으면 Base 것 실행  
  
    function greet() public override  
        returns (string memory) {  
        return "Hello from Child";  
    }  
  
    function adminAction() public  
        onlyOwner {  
        // 부모의 modifier 사용  
    }  
}
```

부모의 상태변수·함수·modifier·이벤트가 자식 컨트랙트에 포함된다. child 를 배포하면 Base 코드가 하나의 바이트코드로 합쳐진다.

상속 (2/3) — virtual / override

VIRTUAL & OVERRIDE — SOLIDITY 0.6.0+ 필수

키워드	위치	의미	없으면
<code>virtual</code>	부모 함수	"자식이 이 함수를 재정의해도 된다" — 재정의 허락	자식이 override 시 컴파일 오류
<code>override</code>	자식 함수	"부모의 그 함수를 내가 덮어쓴다" — 의도 명시	같은 이름 선언 → 컴파일 오류
<code>override(B,C)</code>	다중 상속	두 부모 모두에 있는 함수를 재정의 — 괄호 내 전부 나열	나열 누락 → 컴파일 오류

핵심 규칙

- 부모에 `virtual` 없으면 자식이 override 불가
- 자식 override 시 가시성(visibility)은 동일 이상이어야 함
- 0.6.0부터 두 키워드 모두 의무 — 실수 override 방지

- "덮어쓴다"는 부모 코드를 지우는 게 아님
- 부모 코드는 그대로 — **같은 이름에서 자식이 우선 실행**
- `abstract` 컨트랙트는 미구현 함수 있을 때 사용

이유 — 배포 후 수정 불가 + 자금 사고 방지. 실수로 부모의 핵심 함수를 덮어쓰는 버그를 문법 차원에서 막는다.

상속 (3/3) — super 활용

SUPER KEYWORD — CALLING PARENT IMPLEMENTATION

super로 부모 구현 호출

```
contract Child is Base {  
    function greet() public override  
        returns (string memory) {  
        string memory p = super.greet();  
        // p = "Hello from Base"  
        return string.concat(p, " + Child");  
        // → "Hello from Base + Child"  
    }  
}
```

OpenZeppelin 실전 패턴

```
// OZ의 super._update(...) 패턴  
function _update(...) internal override  
    whenNotPaused {  
    super._update(...);  
    // 부모 로직 먼저 → 자식 로직 추가  
}  
  
// KyoboNFT.sol 실제 코드  
function supportsInterface(...)  
    public view override(  
        ERC1155Upgradeable,  
        AccessControlUpgradeable)  
    returns (bool) {  
    return super.supportsInterface(...);  
}
```

다중 상속에서 super — MRO(C3 선형화) 순서대로 호출된다. super 는 "부모"가 아니라 "MRO 체인에서 내 다음 컨트랙트"를 뜻한다.

02

다중 상속

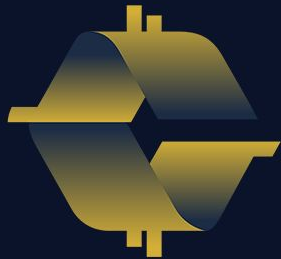
Diamond Problem · C3 선형화 · MRO

Solidity는 is 목록을 오른쪽부터 읽어 선형화한다

다중 is

C3 MRO

Diamond



COINCRAFT
ACADEMY

다중 상속 (1/2) — Diamond 구조 코드

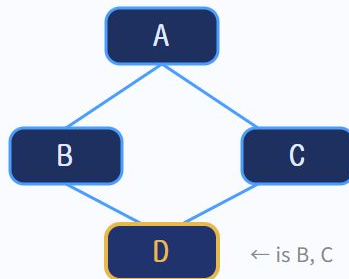
MULTIPLE INHERITANCE — DIAMOND PATTERN

A · B · C — 부모 컨트랙트

```
contract A {  
  function who() public virtual  
    returns (string memory) {  
    return "A";  
  }  
}  
  
contract B is A {  
  function who() public virtual override  
    returns (string memory) {  
    return "B";  
  }  
}  
  
contract C is A {  
  function who() public virtual override  
    returns (string memory) {  
    return "C";  
  }  
}
```

D — 다중 상속 컨트랙트

```
// is B, C — B와 C를 동시에 상속  
contract D is B, C {  
  function who() public  
    override(B, C)  
    returns (string memory) {  
    return super.who(); // "C" 반환  
  }  
}
```



override(B, C) — 두 부모 모두 나열 필수. 하나라도 빠지면 컴파일 오류.

다중 상속 (2/2) — C3 선형화

C3 LINEARIZATION — METHOD RESOLUTION ORDER (MRO)

Diamond Problem 상황

- D가 B와 C를 동시에 상속 → `who()` 가 여러 군데 존재
- `super.who()` 를 부르면 누구 것을 부를지 순서 결정 필요
- 모름모를 한 줄로 펴는 것이 선형화

```
contract D is B, C
```

MRO 순서: **D → C → B → A**

is 목록을 거꾸로 읽으면 가까운 순서

한 줄로 펴는 규칙 (이것만 기억)



- `is` 목록 오른쪽이 더 기본(base)
- `is` 목록 왼쪽이 더 파생(derived)
- `super`는 `is` 목록의 뒤(오른쪽)부터 호출
- A는 단 한 번만 실행 → Diamond 문제 해결

한 줄 요약 — `super` 는 `is` 목록의 뒤(오른쪽)부터 부른다. `contract D is B, C` 에서 `super`는 `C → B → A` 순.

KyoboNFT 다중 상속 구조

KYOBONFT — MULTIPLE INHERITANCE IN PRACTICE

KyoboNFT.sol 선언

```
contract KyoboNFT is
    Initializable,
    ERC1155Upgradeable,
    AccessControlUpgradeable,
    PausableUpgradeable,
    UUPSUpgradeable
{
    // 세 개념이 전부 등장:
    // 상속 + 다중 상속 + 인터페이스 + 라이브러리
    using Strings for uint256;
}
```

각 부모의 역할

부모 컨트랙트	역할
Initializable	initialize() 한 번만 실행 보장
ERC1155Upgradeable	NFT 다중 토큰 표준 구현 (abstract)
AccessControlUpgradeable	역할 기반 접근 제어 (MINTER_ROLE 등)
PausableUpgradeable	긴급 정지 기능
UUPSUpgradeable	프록시 업그레이드 패턴

M7에서 반드시 마주칠 충돌 — ERC1155와 AccessControl이 둘 다 `supportsInterface` 를 가져 Diamond 충돌 발생 → `override(ERC1155Upgradeable, AccessControlUpgradeable)` 로 해결

03

추상 컨트랙트

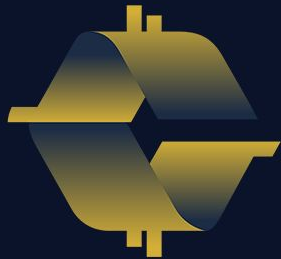
abstract contract

구현된 함수와 미구현 함수를 함께 가질 수 있다

abstract

직접 배포 불가

OZ 패턴



COINCRAFT
ACADEMY

추상 컨트랙트 (1/2)

ABSTRACT CONTRACT — PARTIAL IMPLEMENTATION

abstract contract BaseToken

```
abstract contract BaseToken {
    string public name;
    uint256 public totalSupply;

    constructor(string memory _name) {
        name = _name;
    }

    // 미구현 → 자식이 반드시 구현해야 함
    function _mint(address to, uint256 amt)
        internal virtual;

    // 구현됨 → 자식이 그대로 사용 가능
    function decimals() public pure virtual
        returns (uint8) { return 18; }
}
```

특징 정리

- `abstract` 컨트랙트는 직접 배포 불가
- 반드시 상속해서 미구현 함수를 채워야 배포 가능
- 구현된 함수와 미구현 함수를 함께 가질 수 있음
- 상태변수·constructor·modifier 모두 가능
- 인터페이스와 일반 컨트랙트의 중간 형태

직접 배포 시도 → `TypeError: Contract "BaseToken" should be marked as abstract.`

추상 컨트랙트 (2/2) — 구현 상속

ABSTRACT CONTRACT — IMPLEMENTATION & OPENZEPELIN PATTERN

KyoboToken is BaseToken

```
contract KyoboToken is BaseToken {
    mapping(address => uint256) private _bal;

    constructor()
        BaseToken("KyoboToken") {}

    // 미구현 함수 채우기 → 배포 가능
    function _mint(address to, uint256 amt)
        internal override {
        _bal[to] += amt;
        totalSupply += amt;
    }
}
```

OpenZeppelin ERC1155가 abstract인 이유

- OZ ERC1155.sol / ERC1155Upgradeable.sol 자체가 `abstract`
- 모든 함수가 구현돼 있음에도 abstract — 이유가 다름
- "직접 배포하지 말고 상속해서 mint를 추가하라"는 의도
- 설계 의도로 abstract를 붙인 케이스

결론 — 우리가 ERC1155를 상속받아 `mint()` 로직을 더해 쓰는 이유가 이것이다. KyoboNFT가 그 자식 컨트랙트.

04

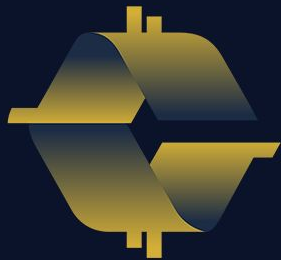
인터페이스

interface · ERC 표준 · 외부 컨트랙트 호출
구현 없이 함수 시그니처와 이벤트만 선언한다

interface

ERC 표준

VASP 연동



COINCRAFT
ACADEMY

인터페이스 (1/4) — IERC20 선언

INTERFACE — FUNCTION SIGNATURES & EVENTS ONLY

interface IERC20

```
interface IERC20 {  
    // 구현 없음, 모두 external  
    function totalSupply()  
        external view returns (uint256);  
    function balanceOf(address account)  
        external view returns (uint256);  
    function transfer(address to, uint256 amt)  
        external returns (bool);  
    function approve(address spender, uint256 amt)  
        external returns (bool);  
    function allowance(address owner, address sp)  
        external view returns (uint256);  
    function transferFrom(...)  
        external returns (bool);  
    // 이벤트 선언 가능  
    event Transfer(address indexed from, ...);  
    event Approval(address indexed owner, ...);  
}
```

인터페이스 특성

- 함수 구현 없음 — 시그니처만 선언
- 모든 함수는 **external** (암묵적으로 virtual)
- 상태변수 선언 불가
- constructor 불가
- modifier 불가
- 이벤트 선언은 가능

ERC 표준 — ERC-20·ERC-721·ERC-1155 모두 이 방식으로 정의된다. 구현은 자유, 규약(함수 시그니처)만 강제.

인터페이스 (2/4) — 구현

INTERFACE IMPLEMENTATION — contract MyToken is IERC20

contract MyToken is IERC20

```
contract MyToken is IERC20 {
    mapping(address => uint256) private _bal;
    uint256 private _total;

    // interface 함수는 암묵적 virtual
    // 구현하는 쪽이 override 불임
    function totalSupply() external view
        override returns (uint256) {
            return _total;
        }
    function transfer(address to, uint256 amt)
        external override returns (bool) {
        require(_bal[msg.sender] >= amt);
        _bal[msg.sender] -= amt;
        _bal[to] += amt;
        emit Transfer(msg.sender, to, amt);
        return true;
    }
    // ... 나머지 함수도 구현 필수
}
```

구현 규칙

- 인터페이스의 모든 함수를 구현해야 함
- 미구현 함수 있으면 → 컴파일 오류
- 구현 시 `override` 키워드 필수
- 가시성은 external 이상 (public도 가능)

미구현 시 → `TypeError: Contract "MyToken" should be marked as abstract because it does not implement all functions.`

인터페이스 구현도 `is` 키워드 사용 — Java의 `implements`와 달리 Solidity는 키워드가 하나.

인터페이스 (3/4) — interface vs abstract

INTERFACE VS ABSTRACT CONTRACT — COMPARISON

항목	interface	abstract contract
상태변수	불가	가능
구현된 함수	불가	가능
constructor	불가	가능
modifier	불가	가능
이벤트 선언	가능	가능
직접 배포	불가	불가
주용도	표준 ABI 정의 · 외부 컨트랙트 호출	부분 구현 + 확장 포인트 제공

interface 선택 — ERC 표준 정의, 외부 컨트랙트 호출, 구현 코드 없어 배포 크기에 기여 없음.

abstract 선택 — 여러 자식이 공통 로직·상태변수·modifier를 공유할 때, OZ ERC20, ERC1155가 이 방식.

인터페이스 (4/4) — 외부 컨트랙트 호출

INTERFACE AS CALL PATTERN — VASP INTEGRATION

외부 컨트랙트 호출 패턴

```
// 배포된 컨트랙트 주소만 있으면 호출 가능
IERC20 token = IERC20(0xAbCd...);
uint256 bal = token.balanceOf(msg.sender);
token.transfer(recipient, 100);

// KyoboNFT 호출 — VASP 서버 코드
IKyoboNFT nft = IKyoboNFT(kyoboNFTAddress);
nft.mint(userAddress, tokenId, amount);

// 소스코드 없어도 됨 — ABI만 있으면 호출
// selector = keccak256("mint(address,uint256,uint256)")[:4]
```

VASP가 인터페이스만 보고 호출하는 이유

- VASP는 KyoboNFT 소스코드를 몰라도 됨
- **IKyoboNFT** 인터페이스(시그니처+이벤트)만 있으면 ABI 생성
- ABI로 함수 선택자 + 인자 인코딩 자동 처리
- 배포 크기 증가 없음 — interface는 바이트코드 0 기어

Phase 1 핵심 — VASP 어댑터가 KyoboNFT를 호출하는 방식이 이것이다. `IKyoboNFT(address).mint(to, tokenId, amount)`

ERC-165 supportsInterface — 런타임에 "이 컨트랙트가 IERC20을 구현하나?" 확인 가능. 잘못된 주소로 호출 시 revert 방지.

05

라이브러리

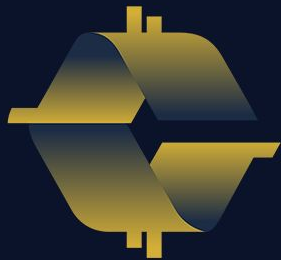
library · using A for B

상태 없는 재사용 가능한 함수 집합

library

using A for B

Strings



COINCRAFT
ACADEMY

라이브러리 (1/2) — using A for B

LIBRARY — ATTACHING FUNCTIONS TO A TYPE

using Strings for uint256

```
import "@openzeppelin/contracts/utils/Strings.sol";

contract TokenURI {
    // uint256 타입에 Strings 함수 붙이기
    using Strings for uint256;

    function uri(uint256 tokenId)
        public pure returns (string memory) {
        return string.concat(
            "https://api.kyobo.com/nft/",
            tokenId.toString() // ← 핵심
            // Strings.toString(tokenId)와 동일
            // 점 앞 값이 첫 번째 인자가 됨
        );
    }
}
```

using A for B 문법

- using Strings for uint256 — uint256 타입에 Strings 라이브러리 함수를 붙임
- tokenId.toString() → Strings.toString(tokenId) 와 동일
- 점(.) 앞의 값이 라이브러리 함수의 첫 번째 인자
- 타입이 맞아야 함 — uint256에만 Strings 함수 사용 가능

KyoboNFT에서 쓰는 이유 — tokenId(숫자)를 URI 문자열에 삽입하기 위해. uri() 함수에서 tokenId.toString() 호출.

```
// KyoboNFT.sol 실제 uri() 함수
function uri(uint256 tokenId) public view override
    returns (string memory) {
    return string.concat(
        _baseURIStorage, tokenId.toString(), ".json");
}
```

라이브러리 (2/2) — internal vs external

LIBRARY TYPES — INLINING VS DELEGATECALL

구분	internal 라이브러리	external 라이브러리
배포	별도 배포 불필요	별도 배포 필요
삽입 방식	컴파일 시 바이트코드에 인라이닝	런타임 DELEGATECALL
gas	외부 CALL 없음 → 저렴	DELEGATECALL 비용 추가
24KB 한도	컨트랙트 크기에 포함	별도 주소 → 분산 가능
사용 빈도	압도적 다수 (OZ 포함)	드물게 사용
예시	OZ Strings, Math, SafeCast	복잡한 계산 라이브러리

실습 라이브러리 — 모두 internal 함수만 사용. OZ의 Strings.toString() 도 internal.

라이브러리 목적 — gas 절약보다 코드 재사용과 24KB 배포 크기 한도 분산이 주목적이다.

06

receive() / fallback()

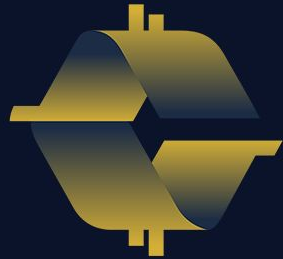
ETH 수신 처리

컨트랙트가 ETH를 받을 때 실행되는 특수 함수

receive

fallback

payable



COINCRAFT
ACADEMY

receive() / fallback() (1/2) — ETH 수신

RECEIVE & FALLBACK — ETH TRANSFER HANDLING

contract Vault

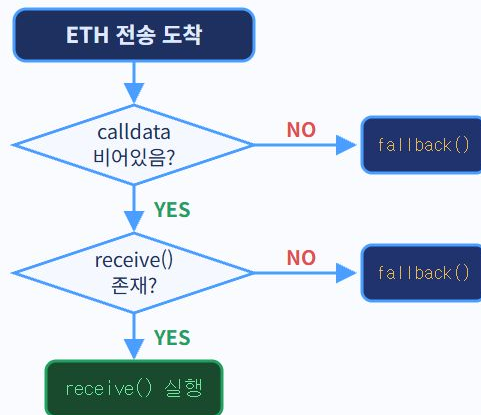
```
contract Vault {
    address public owner;
    event Received(address sender, uint256 amt);

    constructor() { owner = msg.sender; }
    modifier onlyOwner() {
        require(msg.sender == owner); _;
    }

    // 순수 ETH 전송 (data 없음) 시 실행
    receive() external payable {
        emit Received(msg.sender, msg.value);
    }

    // ABI에 없는 함수 호출 or receive 없을 때
    fallback() external payable {
        revert("Unknown function");
    }
}
```

언제 어떤 함수가 호출되는가



KyoboNFT는 ETH 직접 수신 안 함 → receive/fallback 없음.
Gnosis Safe(M9)에서 다시 등장.

receive() / fallback() (2/2) — 출금 패턴

WITHDRAW PATTERN — call vs transfer vs send

withdraw — 권한 검사 필수

```
// ⚠️ onlyOwner 없으면 누구나 잔고 탈취
function withdraw(uint256 amt)
public onlyOwner {
  // ✅ 현재 권장 방식
  (bool ok, ) = payable(msg.sender)
    .call{value: amt}("");
  require(ok, "Withdraw failed");
}
```

call vs transfer vs send

방식	gas 전달	실패 처리
<code>.call{value:}</code>	남은 gas 전체	bool 반환 → require 필수
<code>.transfer()</code>	2300 gas만	실패 시 자동 revert
<code>.send()</code>	2300 gas만	bool 반환 (확인 안 하면 무시)

2300 gas 함정 — transfer/send는 스마트컨트랙트 지갑 상대로 깨질 수 있음. `.call` 사용 권장.

재진입 공격 주의 — `.call` 사용 시 ReentrancyGuard 또는 Checks-Effects-Interactions 패턴 필수.

07

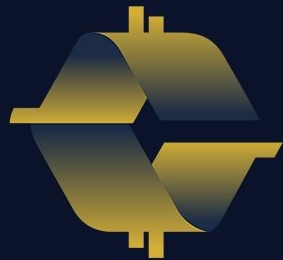
Phase 1 연결

KyoboNFT.sol — 오늘 배운 세 개념의 조합
상속 + 인터페이스 + 라이브러리가 전부 등장한다

KyoboNFT

VASP 흐름

M7 예고



COINCRAFT
ACADEMY

Phase 1 연결 (1/3) — KyoboNFT.sol 구조

KYOBONFT — THREE CONCEPTS IN ONE CONTRACT

KyoboNFT.sol — 세 개념의 조합

KyoboNFT.sol

is ERC1155Upgradeable ← 상속 (abstract 구현)
is AccessControlUpgradeable ← 상속 (다중 상속)
is PausableUpgradeable ← 상속 (다중 상속)
is UUPSUpgradeable ← 상속 (다중 상속)

IKyoboNFT.sol ← 인터페이스
(VASP가 이것만 보고 호출)

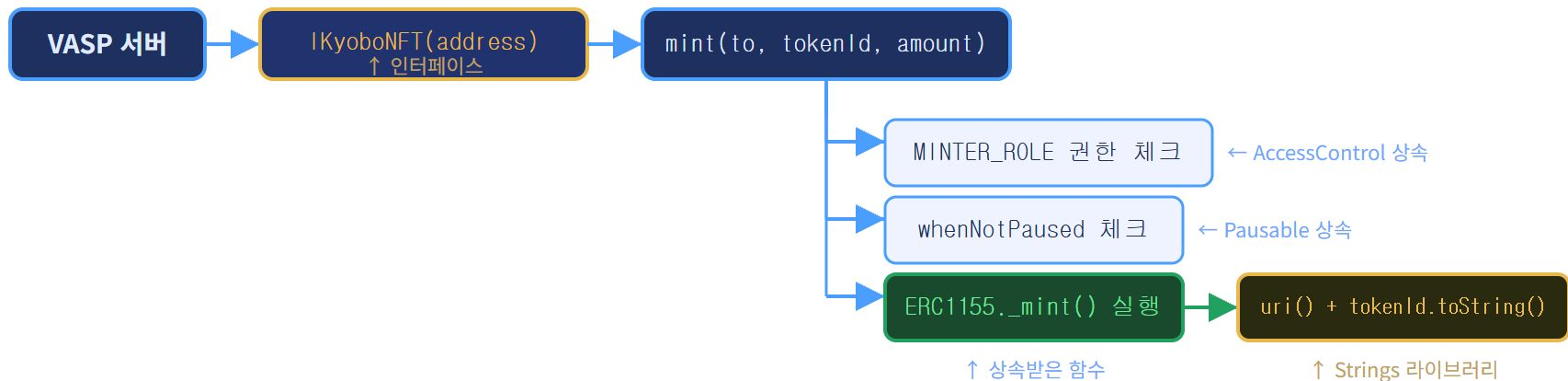
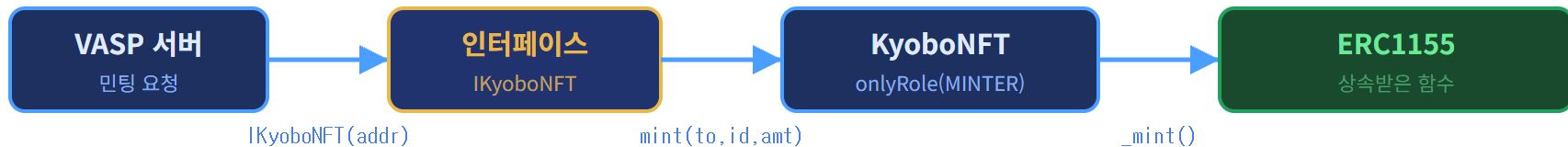
using Strings for uint256 ← 라이브러리
(tokenId → URI 문자열 변환)

각 개념의 역할

개념	KyoboNFT에서의 역할
상속	ERC1155·AccessControl·UUPS 기능 전부 포함
다중 상속	세 부모 동시 상속 — MRO로 충돌 해결
인터페이스	VASP가 소스 없이 ABI만으로 mint 호출
라이브러리	tokenId.toString()으로 URI 조합
추상 컨트랙트	ERC1155Upgradeable(abstract) 상속 후 mint 추가

Phase 1 연결 (2/3) — 호출 흐름

END-TO-END FLOW — VASP → KYOBONFT → NFT ISSUED



핵심 — 오늘 배운 세 개념(상속·인터페이스·라이브러리)이 하나의 컨트랙트에서 각자 역할을 맡아 작동한다.

Phase 1 연결 (3/3) — M7 예고

M7 PREVIEW — TWO THINGS TO KNOW IN ADVANCE

① supportsInterface 충돌

```
// ERC1155와 AccessControl 둘 다 보유
// → Diamond 충돌 발생

// 해결: 두 부모 모두 override 나열
function supportsInterface(bytes4 interfaceId)
    public view override(
        ERC1155Upgradeable,
        AccessControlUpgradeable)
    returns (bool) {
    return super.supportsInterface(interfaceId);
}

// KyoboNFT.sol 실제 코드 그대로
```

override() 괄호 누락 → 컴파일 오류. M7에서 직접 마주친다.

② _authorizeUpgrade 권한 필수

```
// UUPS는 이 함수를 반드시 구현
// 권한 없으면 누구나 컨트랙트 교체 가능!

function _authorizeUpgrade(
    address newImpl)
    internal override
    onlyRole(UPGRADER_ROLE) {
    // UPGRADER_ROLE만 업그레이드 가능
}
```

③ M7 추가 — Upgradeable은 constructor 대신 `initialize()` 에서 `__X_init()` 호출 (상세는 M7)

08 실습

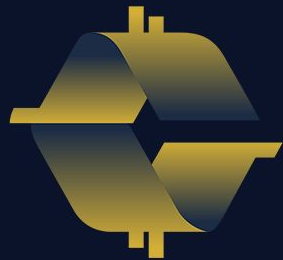
Remix IDE · 10분

상속 + 인터페이스 + 라이브러리 직접 작성

실습 A — 상속

실습 B — 인터페이스

실습 C — 라이브러리



COINCRAFT
ACADEMY

실습 A — 상속 + 다중 상속

EXERCISE A — INHERITANCE · VIRTUAL / OVERRIDE · SUPER

BaseOwnable.sol + SimpleToken.sol

```
contract BaseOwnable {
    address public owner;
    constructor() { owner = msg.sender; }
    modifier onlyOwner() {
        require(msg.sender == owner, "Not owner"); _;
    }
}

contract SimpleToken is BaseOwnable {
    mapping(address => uint256) private _bal;

    function mint(address to, uint256 amt)
        public onlyOwner {
        _bal[to] += amt;
    }

    function balanceOf(address who)
        external view returns (uint256) {
        return _bal[who];
    }
}
```

확인 체크리스트

- 1 mint(Account2, 500) → 정상 실행
- 2 balanceOf(Account2) → 500 반환
- 3 Account1(배포자 아님)이 mint 시도 → "Not owner" revert 확인
- 4 virtual 없이 override 시도 → 컴파일 오류 직접 확인

확인 포인트 — SimpleToken 에 owner 와 onlyOwner 가 없는데도 동작한다. 상속으로 포함됐기 때문.

실습 B — 인터페이스 + 외부 컨트랙트 호출

EXERCISE B — INTERFACE · AT ADDRESS · ABI RESTRICTION

ISimpleToken.sol + TokenReader.sol

```
interface ISimpleToken {
    function balanceOf(address who)
        external view returns (uint256);
    function transfer(address to, uint256 amt)
        external returns (bool);
    event Transfer(address indexed from,
        address indexed to, uint256 amt);
}

contract TokenReader {
    ISimpleToken public token;
    constructor(address addr) {
        token = ISimpleToken(addr);
    }
    function readBalance(address who)
        public view returns (uint256) {
        return token.balanceOf(who);
    }
}
```

확인 순서

- 1 SimpleToken 배포 → `mint(Account2, 500)`
- 2 Account2로 전환 → `transfer(Account3, 200)` → Transfer 이벤트 확인
- 3 SimpleToken 주소로 **TokenReader** 배포 → `readBalance(Account2)` → 300
- 4 Remix "At Address"에 ISimpleToken으로 연결 → **mint 버튼 없음** 확인 ← ABI 제한

핵심 포인트 — TokenReader는 SimpleToken 소스코드를 import하지 않는다. 인터페이스만으로 호출 가능.

실습 C — 라이브러리 + uri() override

EXERCISE C — using Strings for uint256 · uri() IMPLEMENTATION

다이아몬드 MRO 추적

```
contract A {
    event Log(string msg);
    function ping() public virtual {
        emit Log("A.ping"); }
}

contract B is A {
    function ping() public virtual override {
        emit Log("B.before"); super.ping();
        emit Log("B.after"); }
}

contract C is A {
    function ping() public virtual override {
        emit Log("C.before"); super.ping();
        emit Log("C.after"); }
}

// MRO: D → C → B → A
contract D is B, C {
    function ping() public override(B,C) {
        emit Log("D.before"); super.ping();
        emit Log("D.after"); }
}
```

확인 + using Strings 실습

```
// SimpleNFT - uri() 구현
import "@openzeppelin/contracts/utils/Strings.sol";
contract SimpleNFT {
    using Strings for uint256;
    string public baseURI =
        "https://api.kyobo.com/nft/";

    function uri(uint256 tokenId)
        public view returns (string memory) {
        return string.concat(
            baseURI, tokenId.toString(), ".json");
    }
}

// uri(42) → "https://api.kyobo.com/nft/42.json"
```

D.ping() 후 Logs 탭: **D→C→B→A→B(after)→C(after)→D(after)**. A는 단 한 번만 실행됨.

오늘 완료 기준 — virtual/override 컴파일 오류 직접 확인 · super 호출 순서 추적 · 인터페이스 ABI 제한 확인 · tokenId.toString() 작동 확인